Lecture 07

Exceptions, First Class Functions

Exceptions

Exceptions

Exception bindings declare new kinds of exception:

```
exception Bogus
exception BadNum of int
```

raise expressions can throw an exception:

```
raise Bogus
raise (BadNum 1)
```

try … with … expressions can catch exceptions

```
try e with Bogus -> 0

try e with BadNum n -> n
```

Exceptions

Exception bindings declare new kinds of exception:

```
exception Bogus
exception BadNum of int
```

We *build* exceptions, which are “just values” (!) with these constructors

```
Bogus
(BadNum 1)
```

That is different than *raising* an exception with the raise expression:

```
raise Bogus
raise (BadNum 1)
```

A bit more precisely

- New exception bindings add new variants to “the one exception type” **`exn`**
- We build values of type **`exn`** just like we build variant types with constructors
  - Can pass them around, though that isn’t *too* common
- We raise (throw) exceptions with **`raise e`**
  - Type-checking:
    - **`e`** must have type **`exn`**
    - Result type is *any type you want (!)*
  - Evaluation rules:
    - Do not produce a result (!)
    - cancel everything
    - pass the exception produced by **`e`** to the nearest *try expression* on the call stack...

Try/with

`try ... with ...` expressions can *catch* exceptions

```
try e1 with Bogus -> e2
try e1 with BadNum n -> e2
```

Evaluation rules:

- Just evaluate **`e1`** and that is the result
- But if **`e1`** raises an exception *and* that exception *matches* the pattern, then evaluate `e2` and that is the result

Type-checking: **`e1`** and **`e2`** must have the same type and that is the overall type

Group Work

1. Define an exception type `EmptyList`
2. Write your own implementation of `List.hd` that throws an `EmptyList` exception if the list is empty
3. Write a function that takes a list as input and returns 1 + the first value in the list, or returns -100 if the list is empty. Instead of checking whether the list is empty using pattern matching, just call your `hd` function and catch the exception.

First Class Functions

What is *Functional Programming*?

- No crisp definition (“know it when you see it”), but usually includes...
  - Avoiding mutation in most/all cases
  - Using functions as values -- this unit

  … and may include ...
  - Using recursion and recursive variant types
  - Style closer to mathematical definitions
  - Idioms using “laziness” (later topic, briefly)
  - (Bad Definition) Anything not OOP or C ?
- A *functional language* just *encourages* functional programming
  - Often not a clear yes/no

First-class Functions

- First-class means something is “in the language” of expressions -- can compute them, pass them around, put them in data structures, etc.
- Functions in many languages, including OCaml, are first class
  - I never said they weren’t :-)
  - OCaml functions are [almost] values
- Most common use is as an argument to another function
  - Lets us abstract over “what to compute” in certain situations: caller just passes in code to handle!
  - A function that takes or returns functions is called higher-order

```
let double x = x * 2 (* old-fashioned function *)
let incr   x = x + 1 (* old-fashioned function *)

let funcs = [double; incr] (* list of functions *)

let rec apply_funcs (fs, x) =
 match fs with
 | [] -> x
 | f :: fs' -> apply_funcs (fs', f x)

let foo = apply_funcs (funcs, 100) (* 201 *)
let bar = apply_funcs (List.rev funcs, 100) (* 202 *)
```

Function Closures

Environment where the function was defined

- Functions can use bindings from the *enclosing environment*
  - Even if function is passed around and called somewhere else
  - Makes first-class functions *much* more powerful and useful
- Will study this carefully after some simpler examples
  - Will need function *values* to be not just functions, but also the environments where they were defined
  - “Function + environment” is called a *function closure*
- In theory, a language could have 1st-class functions without closures
  - Or vice versa, but typically a language with one has the other

Plan

1. How to use first-class functions and function closures
2. The precise semantics for function closures and function calls
3. Multiple powerful idioms this semantics enables

Functions as Arguments

- Can pass one function as an argument to another function
  - Again, not really a “new feature”, we just haven’t done this before
- Elegant way to factor out common code
  - Replace N similar functions with calls to 1 function with N different (short) function arguments

```
let rec n_times (f,n,x) =
 if n = 0 then
   x
 else
   f (n_times (f, n - 1, x))
```

Types for `n_times`

- What's the type of `n_times`?

```
let rec n_times (f,n,x) =
 if n = 0 then
   x
 else
   f (n_times (f, n - 1, x))
```

Types for `n_times`

- What's the type of **`n_times`**?

```
let rec n_times (f,n,x) =
 if n = 0 then
   x
 else
   f (n_times (f, n - 1, x))
```

- `val n_times : ('a -> 'a) * int * 'a -> 'a`
  - Types are *inferred* based on how args are used!
  - More useful than `(int -> int) * int * int -> int`
  - See lecture code for examples of how to use

Types for **`n_times`**

- What's the type of **`n_times`**?

```
let rec n_times (f,n,x) =
 if n = 0 then
   x
 else
   f (n_times (f, n - 1, x))
```

- `val n_times : (int -> int) * int * int -> int ?`

Relation to Types

- Higher-order functions often “so reusable” that they have polymorphic types, i.e., types with type variables (e.g., `'a`)
- There are higher-order functions that are not polymorphic
  - `val times_until_zero : (int -> int) * int -> int`
- And there are polymorphic functions that are not higher-order
  - `val len : 'a list -> int`
- Always a good idea to understand a function’s type, especially for higher-order functions

Group Work

Give the types of the following expressions (or say it's a type error):

```
let f x = x + 1
let g x = f (f x)
let h (x, y) = (f y, g x)
let a = [ f; g; h ]
let b = [ f; g ]
let m (f1, x1, y1) = f1 x1 + y1
let rec n fs =
 match fs with
 | [] -> 0
 | f :: fs -> f 1 + n fs
```

Map

```
(* ('a -> 'b) -> 'a list -> 'b list *)
let rec map f xs =
  match xs with
  | [] -> []
  | x :: xs' -> f x :: map f xs'
```

Hall-of-fame higher-order function

- Name is standard (used for any sort of collection data structure)
  - Generally provided in standard libraries for standard collections
- Used all the time in functional code, thus *idiomatic*
  - Using it where appropriate isn't just "shorter code" but *communicates what you are doing*
  - Conversely, not using it when "doing a map" confuses your reader

Filter

```
(* ('a -> bool) * 'a list -> 'a list *)
let rec filter (f, xs) =
  match xs with
  | [] -> []
  | x :: xs' -> if f x then
                  x :: filter (f, xs')
                else
                  filter (f, xs')
```

- Another hall-of-famer
  - Keep only some elements from a collection
  - Also an idiom, so you should use whenever “doing a filter”

Anonymous Functions

- Syntax: `fun p -> e`
  - Where `p` is a pattern and `e` is an expression
- Function as an expression!
  - Type checking, evaluation rules “the same” as function bindings
  - Note that use `->` instead of `=` to separate parameters from function body
  - No function name!
  - “Just an expression”- Can appear anywhere expressions allowed!

Using Anonymous Functions

- Usually use anonymous functions as arguments to other functions
  - No need to name a function that you use only once!
- One limitation: anonymous functions cannot call themselves
  - No name to make a recursive call with
- Non-recursive function bindings are really just syntactic sugar!!
  - `let f p = e` really just means `let f = fun p -> e`
  - Again, doesn't work for recursive functions
  - And as usual, better style to use the syntactic sugar available

A Point on Style

- **Bad:** `if e then true else false`
- **Good:** `e`
- **Bad:** `fun x -> f x`
- **Good:** `f`
- **Bad:** `n_times ((fun x -> List.tl x), 3, xs)`
- **Good:** `n_times (List.tl, 3, xs)`

Generalizing

- Our examples so far have been awfully similar
  - Take one function as argument then process a number or list
- We can do much more!
  - Pass several functions as arguments
  - Put functions in data structures
  - Return functions as results
  - Write higher-order functions over variant types
- Useful whenever abstracting over “what to compute with”

Returning functions

```
let double_or_triple f =
 if f 7
 then fun x -> 2 * x
 else fun x -> 3 * x
```

Functions are first-class

- They can be returned from (other) functions
- Silly example above

Has type `(int -> bool) -> (int -> int)`
But REPL reports `(int -> bool) -> int -> int`

- This is the same thing
- REPL never prints unnecessary parentheses
- `t1 -> t2 -> t3 -> t4` means `t1->(t2->(t3->t4))`
- Will learn soon why this *precedence* is convenient

Other data structures

- Higher-order functions are not just for lists
- They work great for common traversals over your own data structures
- See **`true_of_all_constants`** in the code file for one example